Real-Time Object Tracking in C++

Project Report

Banoth Kavya | B.Tech ECE (Industrial IoT), NIT Kurukshetra | October 2026

Stack	C++, OpenCV (CSRT/KCF trackers), CMake, gdb
Type	Individual project
Code	github.com/kavyabanoth

1. Problem Statement

Many applications, such as surveillance, robotics and defence systems, need to follow a chosen object through live video and report the tracking status in real time. This project builds a C++ application that lets a user select a target in a video stream and tracks it frame by frame while displaying live status information.

2. Objectives

- Capture live webcam or recorded video using OpenCV in C++.
- Let the user select the target object interactively with a region-of-interest (ROI) box.
- Track the selected object with OpenCV CSRT/KCF trackers and draw its bounding box live.
- Display frames per second (FPS) and tracking status on screen.
- Build with CMake, debug with gdb, and document a test procedure.

3. Approach and Implementation

Step	Description
1. Video input	The program opens a webcam or video file with OpenCV's video capture class and reads frames in a loop.
2. ROI selection	On the first frame, the user draws a box around the target. This box initialises the tracker.
3. Tracking	For every new frame the CSRT/KCF tracker predicts the object's new bounding box and reports whether tracking succeeded.
4. Live display	The bounding box, current FPS and tracking status (tracking / lost) are drawn on each frame in a GUI window.
5. Build and debug	The project is built with CMake using g++, and runtime issues are debugged with gdb.

4. Test Procedure

Run each test and confirm the expected behaviour.

#	Test	Expected behaviour
1	Start the program with a webcam	Live video window opens without errors.
2	Select an object with the ROI box	Bounding box appears around the object and tracking starts.
3	Move the object slowly across the frame	Box follows the object; status shows tracking.
4	Move the object quickly	Box follows, or status changes to lost if the tracker cannot keep up.

#	Test	Expected behaviour
5	Partly cover the object, then reveal it	Tracker may drift or lose the object; behaviour is noted.
6	Observe the on-screen FPS	FPS value updates continuously while tracking.
7	Reselect the target and quit using the program keys	Tracker restarts on the new target; the program exits cleanly.

5. Known Limitations of This Approach

- Single-object tracking only; the target must be selected manually at the start.
- Trackers like KCF and CSRT can drift or fail under heavy occlusion, fast motion or large changes in scale and lighting.
- Speed depends on the processor, since no GPU acceleration is used.

6. Future Work

- Add automatic detection (colour thresholding or a detection model) so tracking can start and recover without manual selection.
- Track multiple objects at once and compare different tracker types on speed and accuracy.
- Port the pipeline to an embedded board and drive hardware based on the tracking result.

7. Conclusion

The project demonstrates a complete real-time visual tracking pipeline in C++ using OpenCV: video capture, interactive target selection, tracking and live status display, built with CMake and debugged with gdb. It gave hands-on practice with C++ image-processing code, the build toolchain and testing against a written procedure.